

Les animations

1. Introduction aux Animations Flutter

Les animations sont un élément essentiel de toute application mobile réussie, car elles améliorent l'expérience utilisateur en rendant les interactions plus agréables et intuitives. Flutter offre un large éventail d'outils et de widgets pour créer des animations captivantes, allant des transitions simples aux animations complexes et interactives.

Dans cette introduction, nous explorerons les principales fonctionnalités d'animation de Flutter, les différents types d'animations que vous pouvez créer, ainsi que les avantages de l'utilisation de Flutter pour donner vie à vos designs d'interface utilisateur. Que vous soyez un développeur débutant ou expérimenté, Flutter offre des moyens flexibles et puissants pour intégrer des animations dans vos applications, et nous allons plonger dans ce sujet passionnant pour vous donner un aperçu complet de ses possibilités.

2. AnimatedContainer

`AnimatedContainer` est une classe Flutter qui hérite des fonctionnalités de base du widget `Container`, mais avec une capacité d'animation intégrée. Il permet de créer des animations de transition fluides lorsque les propriétés telles que la taille, la couleur, la marge, le rembourrage, etc., de votre conteneur changent. Cela peut ajouter une touche de dynamisme à votre application Flutter.

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: MyAnimatedContainer(),
    );
  }
}
```

```

class MyAnimatedContainer extends StatefulWidget {
  @override
  _MyAnimatedContainerState createState() => _MyAnimatedContainerState();
}

class _MyAnimatedContainerState extends State<MyAnimatedContainer> {
  bool _isRed = true;

  void _toggleColor() {
    setState(() {
      _isRed = !_isRed;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('AnimatedContainer Example'),
      ),
      body: Center(
        child: GestureDetector(
          onTap: _toggleColor,
          child: AnimatedContainer(
            duration: Duration(seconds: 1),
            width: 200,
            height: 200,
            color: _isRed ? Colors.red : Colors.blue,
            child: Center(
              child: Text(
                'Tap to change color',
                style: TextStyle(color: Colors.white),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

```

Dans cet exemple, nous avons un `AnimatedContainer` qui change sa couleur de fond de rouge à bleu lorsque l'utilisateur appuie dessus. L'animation est gérée automatiquement grâce à la propriété `duration` que nous avons définie.

3. Tween Animation

Les animations "tween" (abréviation de "in-between" en anglais) sont un moyen courant d'animer des objets ou des éléments graphiques de manière fluide en interpolant (calculant les valeurs intermédiaires) entre deux états clés. Ces animations sont souvent utilisées dans le développement d'applications mobiles, de jeux vidéo et de sites web pour créer des transitions visuellement attrayantes et des mouvements fluides. Dans ce cours, je vais vous expliquer les concepts de base des animations tween et comment les utiliser.

Concepts de base des animations tween :

1. **États clés (keyframes)** : Les animations tween sont basées sur des états clés, également appelés keyframes. Ce sont des moments précis dans l'animation où vous spécifiez la position, la taille, la rotation, la couleur, ou d'autres propriétés de l'objet que vous souhaitez animer.
2. **Interpolation** : L'interpolation est le processus de calcul des états intermédiaires entre deux états clés. Par exemple, si vous avez un objet à l'état initial A et un état final B, l'interpolation calcule les états intermédiaires pour créer une transition fluide de A à B.
3. **Courbe d'interpolation** : Les courbes d'interpolation, également appelées courbes de transition ou d'easing, déterminent comment l'objet se comporte pendant l'animation. Les courbes peuvent être linéaires (constantes), accélérées (ease-in), décélérées (ease-out), ou personnalisées pour obtenir des effets spécifiques.
4. **Durée de l'animation** : La durée de l'animation détermine combien de temps l'objet prend pour passer d'un état clé à un autre. Une durée plus longue crée des animations plus lentes, tandis qu'une durée plus courte crée des animations plus rapides.
5. **Répétition et boucle** : Vous pouvez spécifier si une animation doit se répéter un certain nombre de fois ou si elle doit boucler en continu.

```

import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: MyAnimatedWidget(),
    );
  }
}

class MyAnimatedWidget extends StatefulWidget {
  @override
  _MyAnimatedWidgetState createState() => _MyAnimatedWidgetState();
}

class _MyAnimatedWidgetState extends State<MyAnimatedWidget>
  with SingleTickerProviderStateMixin {
  late AnimationController _controller;
  late Animation<double> _animation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(
      vsync: this,
      duration: Duration(seconds: 2),
      // Courbe d'interpolation
      curve: Curves.easeInOut,
    );

    _animation = Tween<double>(
      begin: 0.0,
      end: 1.0,
    ).animate(_controller);

    _controller.forward(); // Démarrer l'animation
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Animation Tween en Flutter'),
    ),
    body: Center(
      child: FadeTransition(
        opacity: _animation,
        child: FlutterLogo(
          size: 200.0,
        ),
      ),
    ),
  );
}

@override
void dispose() {
  _controller.dispose(); // Libérer les ressources du contrôleur
  super.dispose();
}

```

Dans cet exemple, nous utilisons une animation tween pour animer l'opacité d'un widget `FlutterLogo` en utilisant `FadeTransition`. Le contrôleur d'animation démarre l'animation lorsque le widget est créé, et l'opacité du logo change en fonction de la valeur de l'animation.

Les animations Tween en Flutter offrent un moyen puissant et flexible d'ajouter des animations à votre application, ce qui la rend plus interactive et attrayante pour les utilisateurs. Vous pouvez personnaliser ces animations pour répondre à vos besoins spécifiques.

4. Implicit Animations

les "implicit animations" (animations implicites) font référence à un ensemble de widgets et de fonctionnalités qui permettent de créer des animations de manière plus simple et intuitive, sans avoir à gérer manuellement les détails complexes de l'animation.

L'idée derrière les animations implicites est de simplifier le processus de création d'animations en encapsulant les détails techniques, tels que la gestion des valeurs de début et de fin, la gestion des chronologies, et la création d'interpolations, dans des widgets prêts à l'emploi. Cela permet aux développeurs de se concentrer sur la définition de ce qu'ils veulent animer plutôt que sur la façon de le faire.

Les animations implicites sont généralement utilisées pour animer des propriétés telles que la taille, la position, la couleur, la transparence, etc. Voici quelques-uns des widgets et des fonctionnalités couramment utilisés pour créer des animations implicites en Flutter :

1. **AnimatedContainer**: Ce widget permet d'animer automatiquement les changements de propriétés tels que la largeur, la hauteur, la couleur, la marge, etc., lorsqu'il est reconstruit avec de nouvelles valeurs.
2. **AnimatedOpacity**: Utilisé pour animer la transparence (opacité) d'un widget lorsque la valeur de l'opacité change.
3. **Tween**: Les objets Tween définissent une plage de valeurs entre lesquelles vous souhaitez animer une propriété. Ils sont couramment utilisés avec d'autres widgets d'animation pour spécifier la plage de valeurs de début et de fin.
4. **ImplicitlyAnimatedWidget**: Il s'agit d'une classe de base pour créer des widgets personnalisés avec des animations implicites. Vous pouvez l'utiliser pour créer vos propres animations implicites personnalisées.
5. **AnimatedSwitcher**: Ce widget facilite le remplacement en douceur d'un enfant par un autre avec une animation implicite lorsqu'un nouvel enfant est ajouté ou supprimé.
6. **AnimatedBuilder**: Ce widget vous permet de construire des animations plus complexes en utilisant un contrôleur d'animation personnalisé tout en tirant parti de la gestion d'état d'animation automatique.
7. **TweenAnimationBuilder**: Similaire à AnimatedBuilder, mais spécialement conçu pour travailler avec des objets Tween.

Pour utiliser les animations implicites, vous devrez généralement fournir un état initial et un état final, puis laisser Flutter gérer le reste. Cela peut se faire en utilisant des widgets tels que Tween, AnimatedContainer, AnimatedOpacity, etc., en fonction de la propriété que vous souhaitez animer.

En résumé, les animations implicites en Flutter simplifient le processus de création d'animations en automatisant de nombreux aspects techniques. Elles sont particulièrement utiles pour rendre vos interfaces utilisateur plus interactives et attrayantes sans avoir à écrire beaucoup de code d'animation personnalisé.

5. Hero Animation

C'est une technique d'animation qui permet de créer des transitions fluides et visuellement agréables entre deux widgets dans deux écrans différents. Cette animation est couramment utilisée pour créer des transitions entre des listes d'éléments et des pages détaillées, où un élément de la liste est agrandi pour remplir l'écran de destination.

Le concept de l'animation de héros repose sur l'idée que deux widgets portant le même tag héros (Hero) peuvent être animés de manière synchronisée entre deux écrans. Voici comment cela fonctionne en quelques étapes :

1. **Widget Source (écran d'origine)** : Vous avez un widget dans votre écran d'origine (par exemple, une miniature d'image) que vous souhaitez agrandir et animer vers un nouvel écran.
2. **Widget Destination (écran de destination)** : Vous avez un widget correspondant dans l'écran de destination (par exemple, une image en plein écran) où le widget source devra être animé.
3. **Tag héros (Hero tag)** : Vous attribuez le même tag héros (Hero tag) aux deux widgets, c'est ce qui permet à Flutter de comprendre que ces widgets sont liés et peuvent être animés ensemble.
4. **Transition** : Lorsque vous naviguez de l'écran source à l'écran de destination, Flutter détecte le tag héros correspondant et anime de manière fluide le widget source vers sa position dans l'écran de destination.

Code dart

```
// Écran d'origine
Hero(
  tag: "imageTag", // Le même tag doit être utilisé dans l'écr
```

```

    an de destination
    child: GestureDetector(
      onTap: () {
        Navigator.of(context).push(MaterialPageRoute(
          builder: (context) => DestinationScreen(),
        ));
      },
      child: Image.asset('assets/thumbnail.png'),
    ),
  ),
)

// Écran de destination
Scaffold(
  body: Hero(
    tag: "imageTag", // Le même tag utilisé dans l'écran d'origine
    child: Image.asset('assets/full_image.png'),
  ),
)

```

Dans cet exemple, lorsque l'utilisateur appuie sur l'image miniature dans l'écran d'origine, Flutter animera en douceur cette image vers l'écran de destination où elle sera affichée en plein écran.

L'animation de héros est un moyen efficace d'améliorer l'expérience utilisateur en rendant les transitions entre les écrans plus agréables et engageantes.

6. Page Transition Animations

Les "page transition animations" (animations de transition de page) en Flutter font référence à la manière dont une nouvelle page ou écran s'anime lorsqu'elle est affichée, généralement lors de la navigation entre différentes parties de votre application. Ces animations sont essentielles pour rendre l'expérience utilisateur plus fluide et attrayante, en aidant les utilisateurs à comprendre la relation spatiale entre les différentes vues de l'application. Flutter propose différentes manières de personnaliser ces animations pour répondre aux besoins spécifiques de votre application.

Voici quelques-unes des animations de transition de page couramment utilisées en Flutter :

1. **PageRouteBuilder** : Cette classe vous permet de personnaliser les transitions de page en utilisant un `PageRouteBuilder`. Vous pouvez spécifier des animations telles que les transitions de fondu, de déplacement ou d'échelle.
2. **Hero Animation** : Comme mentionné précédemment, les animations de héros sont une technique courante pour les transitions entre pages. Elle consiste à animer un widget d'une page à l'autre de manière fluide et élégante, généralement avec un effet de fondu ou de zoom.
3. **PageRoute** : Flutter propose également des transitions de page prédéfinies telles que `MaterialPageRoute`, qui utilise les transitions de page spécifiques au Material Design, telles que le glissement horizontal lors de l'ouverture d'une nouvelle page.
4. **Custom Page Transitions** : Vous pouvez créer des transitions de page entièrement personnalisées en combinant diverses animations, telles que des animations de déplacement, d'opacité, de rotation, etc., en utilisant `PageRouteBuilder` pour créer une transition sur mesure.

Voici un exemple simple d'utilisation de `PageRouteBuilder` pour créer une animation de transition de page de fondu :

```
dartCopy code
Navigator.push(
  context,
  PageRouteBuilder(
    pageBuilder: (context, animation, secondaryAnimation) {
      return SecondPage(); // Page de destination
    },
    transitionsBuilder: (context, animation, secondaryAnimation, child) {
      const begin = Offset(1.0, 0.0);
      const end = Offset.zero;
      const curve = Curves.easeInOut;
      var tween = Tween(begin: begin, end: end).chain(CurveTween(curve: curve));
      var offsetAnimation = animation.drive(tween);

      return SlideTransition(
        position: offsetAnimation,
        child: child,
      );
    },
  ),
);
```

Dans cet exemple, nous utilisons `PageRouteBuilder` pour personnaliser la transition de page en utilisant une animation de glissement horizontal (`SlideTransition`) de la droite vers la gauche. Vous pouvez personnaliser davantage ces animations pour créer des effets uniques en fonction de vos besoins.

En résumé, les "page transition animations" en Flutter sont des animations qui se produisent lors de la navigation entre différentes pages ou écrans de votre application. Vous pouvez les personnaliser en utilisant diverses méthodes pour créer des transitions attrayantes et fluides qui améliorent l'expérience utilisateur globale de votre application.

7. Physics-Based Animation

Un "physics simulator" en Flutter peut être utilisé pour créer des jeux, des animations interactives, des simulateurs de physique réels, des démonstrations scientifiques et bien plus encore. Il existe de nombreuses ressources en ligne, des tutoriels et des exemples de code pour vous aider à démarrer avec la création de simulations physiques dans Flutter en fonction de vos besoins spécifiques.

Voici une explication générale de la façon dont vous pouvez mettre en place un "physics simulator" en Flutter :

1. **Comprendre les principes de base de la physique** : Avant de commencer à créer une simulation physique, il est important de comprendre les principes de base de la physique qui seront impliqués. Cela inclut des concepts tels que la gravité, la friction, la cinématique, etc.
2. **Utilisation de packages de simulation physique** : Flutter dispose de plusieurs packages et bibliothèques qui facilitent la création de simulations physiques. L'un des packages les plus couramment utilisés est "flutter_physics". Ce package fournit des classes et des méthodes pour créer des simulations physiques en utilisant la bibliothèque de simulation physique de Flutter.
3. **Création d'objets physiques** : Vous pouvez créer des objets physiques (par exemple, des balles, des boîtes, des particules, etc.) en utilisant les classes fournies par le package de simulation physique. Ces objets auront des propriétés physiques telles que la masse, la vitesse, la position, etc.
4. **Définition des forces** : Pour simuler le comportement physique des objets, vous devez définir les forces qui agissent sur eux. Par exemple, la gravité

est une force courante qui attire les objets vers le bas. Vous pouvez également définir d'autres forces telles que la friction, les forces de collision, etc.

5. **Mise à jour de la simulation** : Dans votre application Flutter, vous devrez mettre à jour la simulation à chaque itération. Cela signifie recalculer la position et la vitesse de chaque objet en fonction des forces qui agissent sur eux. Cela se fait généralement dans une boucle de simulation.
6. **Rendu graphique** : Une fois que la simulation a été mise à jour, vous devez mettre à jour l'interface utilisateur pour refléter les nouvelles positions des objets. Vous pouvez utiliser les valeurs calculées pour mettre à jour la position des éléments graphiques à l'écran.
7. **Gestion des collisions** : Si votre simulation implique des objets en mouvement, vous devrez gérer les collisions entre eux. Vous devrez détecter quand deux objets entrent en collision et prendre des mesures appropriées pour gérer cette collision.
8. **Optimisation et ajustements** : Pour obtenir des performances optimales et une simulation réaliste, vous devrez peut-être ajuster les paramètres de votre simulation, tels que la fréquence de mise à jour, la résolution, etc.
9. **Tests et débogage** : Comme pour tout développement logiciel, vous devrez effectuer des tests et déboguer votre simulation pour vous assurer qu'elle fonctionne correctement.